OFFICE COPY, ROOM 834
Please make comments, suggestions on this copy or net-mail them to KMP.

# MACSYMA
# IMPLEMENTORS' GUIDE

Draft, January 18, 1981

© 1981, Massachusetts Institute of Technology

# Lisp Utilities

# Functional Definition

## FSET

(FSET *sym def*)

[2 args] Tries to emulate Lispm/NIL FSET. Won't work for LSUBRs, FEXPRs, or FSUBRs.

Examples:

```
(FSET 'KAR 'CAR)                    → CAR                    ; Defines KAR
(FSET 'KAR '(LAMBDA (X) (CAR X)))   → (LAMBDA (X) (CAR X))   ; Defines KAR
(FSET 'KAR (GET 'CAR 'SUBR))        → #13514                 ; Defines KAR
```

# List Manipulation

## *MAKE-LIST

(*MAKE-LIST *size* [*val*])

[1 or 2 args] Creates a list *size* long. The object *val* (default ()) is used to fill the lists.

Examples:

```
(*MAKE-LIST 3)     → (() () ())
(*MAKE-LIST 3 'A) → (A A A)
```

# Mapping

## MAP2C

(MAP2C *f l*)

[2 args] *f* is assumed to be a function of two arguments. It is mapped down *l* and applied to consequtive pairs of elements of the list. Useful for iterating over property lists.

# Mapping (Cont'd)

## ANDMAPC

(ANDMAPC *f l*)

[2 args] Predicate returns T if all elements of *l* answer non-() to the given predicate function *f*.
For an open-coded version of this, see EVERY.

Examples:

```
(ANDMAPC #'FIXP '(1 2 3)) → T
(ANDMAPC #'FIXP '(1 2 A)) → ()
```

## ORMAPC

(ORMAPC *f l*)

[2 args] If any element of *l* answers non-() to the given predicate function *f*, that value is returned. If no answers are non-(), () is returned.
For an open-coded version of this, see SOME. Note, however, that its return value is slightly different.

Examples:

```
(ORMAPC #'FIXP '(1 2 A)) → T
(ORMAPC #'FIXP '(A B C)) → ()
```

## ANDMAPCAR

(ANDMAPCAR *f l*)

[2 args] Like MAPCAR, except if an application of *f* to any of the elements of *l* returns (), then the function returns () immediately. If all applications of the function return non-() values, the list of results is returned.

Examples:

```
(ANDMAPCAR #'1+ '(1 2 3))      → (2 3 4)
(ANDMAPCAR #'FIXP '(1 2 A 3)) → ()
```

# Lispy Logic

## XOR

(XOR $x$ $y$)

[2 args] If one of $x$ and $y$ is non-( ), and the other is ( ), returns the non-( ) argument. Else returns ( ).

Examples:

```
(XOR () ()) → ()
(XOR T T)   → T
(XOR 'A ()) → A
(XOR () T)  → T
```

# Set Membership

## AMONG

(AMONG *item l*)

[2 args] A MEMQ which works at all levels of a piece of list structure, returning T if any terminal node of the list structure is eq to *item*.
Note that a second arg of () will always return (), so it should be avoided.

Examples:

```
(AMONG 'A '((B A) C))          → T
(AMONG 'B '(((E F) B G) E))    → T
(AMONG () '(A B C))            → () ; Avoid this
(AMONG () '(A () B))           → () ; Avoid this
```

## AMONGL

(AMONGL *items l*)

[2 args] Similar to AMONG, but takes a list of *items* to look for. If any of the *items* are eq to a terminal node, a non-() value is returned, the car of which is the item that was found. If no such terminal is found, () is returned.
Note that ()'s will not be seen by AMONGL so they should not be put in the *items* list.

Examples:

```
(AMONGL '(A B) '((C D) E))          → ()
(AMONGL '(A Z) '((X Y) A B C))      → (A Z)
(AMONGL '(A Z) '((Z Y X) A B C))    → (Z)
(AMONGL '(() C) '(A () B))          → ()     ; Avoid this
```

# Set Union

## RECONC

(RECONC $l_1$ $l_2$)

[2 args] Like Lisp's NRECONC, but not destructive. Appends $l_2$ to the reverse of $l_1$.

Example:

(RECONC '(A B C) '(D E F)) → (C B A D E F)

# Set Extraction

## NXTTOLAST

Returns the second-to-last element (not cons) of a list.
Note: For implementational reasons, currently works only on even length lists.

Examples:

```
(NXTTOLAST '(A B))          → A
(NXTTOLAST '(A B C D E F)) → E
(NXTTOLAST '(A B C D E))    → E ; Sample lossage
```

## FIRSTN

(FIRSTN *n l*)

[2 args] Returns the first *n* elements of *l*.
A negative *n* does harmless things on the pdp10 and in Franz but on the LispM it creates a rather severe error, so should be avoided.

Examples:

```
(FIRSTN 3 '(A B C D E)) → (A B C)
(FIRSTN 5 '(A B C D))   → (A B C D)
(FIRSTN -2 '(A B C))    → error!
```

# AList Handling

## ASSQR

(ASSQR $x$ $l$)

[2 args] Like ASSQ but checks against the CDR instead of the CAR. Returns first element of $l$ whose cdr is EQ to $x$, or () if no such element is found.

Examples:

```
(ASSQR 'A '((A . C) (B . A))) → (C . A)
(ASSQR 'A '((B . A) (C . A))) → (B . A)
(ASSQR 'A '((B . C) (C . D))) → ()
```

## DOT2L

(DOT2L $l$)

Takes a list in "alist" form and converts it to one in "property list" form. It is an error if all elements of the list are not conses.

Example:

```
(DOT2L '((A . B) (C . D))) → (A B C D)
```

# PList Handling

## A-ATOM

(A-ATOM *sym sel val*)

[3 args] Obsolete form of C-PUT (a function with the same functionality but different argument order).
If *val* is (), the *sel* property of *sym* is flushed. Else, if *val* is non-(), it becomes the value of the *sel* property of *sym*.
This function should not be called directly from anywhere in Macsyma. Some of MRG's macros still expand into it, however. The purpose is to make a property list look a bit more like a data structure.

Equivalent to:

(C-PUT *sym val sel*)

## C-PUT

(C-PUT *sym val sel*)

[3 args] If *val* is (), the *sel* property of *sym* is flushed. Else, if *val* is non-(), it becomes the value of the *sel* property of *sym*.
This function should not be called directly from anywhere in Macsyma. Some of MRG's macros still expand into it, however. The purpose is to make a property list look a bit more like a data structure.

# Math Functions

## LOG2

(LOG2 $x$)

[1 arg] Returns fix($\log_2(x)$) for positive integers and fix($\log_2(-x)$) for negative integers. Not defined on other kinds of numbers.

Examples:

```
(LOG2 3)                              → 1.
(LOG2 4)                              → 2.
(LOG2 7)                              → 2.
(LOG2 12345123451234512345123451.)    → 80.
```

# PName Hacking

## CONCAT

(CONCAT $q_1$ $q_2$ ...)

[0 or more args] Puts the printnames of all the args together as an interned symbol.
Binds *NOPOINT to T and BASE to 10. during the operation. Otherwise the same as the Lisp function SYMBOLCONC.

Examples:

```
(CONCAT 'A 'B)                  → AB
(CONCAT $INCHAR $LINENUM)       → $C3
(CONCAT '$ 'C 13.)              → $C13
(CONCAT '$A '$B)  . . .         → $A$B
```

# Macsyma Utilities

MEMALIKE

(MEMALIKE *item l*)

[2 args] Like MEMBER in Lisp except uses ALIKE1 instead of EQUAL as the predicate.

ASSOL

(ASSOL *item l*)

[2 args] Like ASSOC in Lisp except uses ALIKE1 instead of EQUAL as the predicate.

ASSOLIKE

(ASSOLIKE *item l*)

[2 args] Like (CDR (ASSOC ...)) in Lisp except uses ALIKE1 instead of EQUAL as the predicate.

Equivalent to:

(CDR (ASSOL *item l*))

# I/O System

DISPLA

(DISPLA *mexp*)

[1 arg] Does a left-justified Macsyma display of *mexp* to the default output stream. To alter where output goes, binding or setting Lisp I/O control variables such as OUTFILES, ↑W, and ↑R.

When given an argument of the form ((MLABLE) *label mexp*), DISPLA will display *mexp* centered. If *label* is non-(), it will be displayed left-justified in parentheses. This is how Macsyma C/D/E-line display is implemented.

Given an argument of the form ((MTEXT) $spec_1$ $spec_2$ ...), DISPLA will attempt to display the text left-justified with each *spec* touching the next. This is useful for mixing string messages with math expressions. Nested MTEXT expressions will appear as parenthetical remarks.

Sample Usage:

```
(DISPLA '((MPLUS) $X $Y))
X + Y
()
(DISPLA '((MLABLE) $C1 ((MPLUS) $X $Y)))
(C1) X + Y
()
(DISPLA '((MLABLE) () ((MPLUS) $A $B $C)))
X + Y
()
(DISPLA '((MTEXT) '|&Answer is | ((MPLUS) $X $Y)))
Answer is X+Y
()
```

## MFORMAT

*Special Form* (MFORMAT *stream cstring* $q_1$ $q_2$ ...)

The preferred Macsyma output routine. Prints several arguments to *stream* (*stream* = T defaults to STANDARD-OUTPUT) according to a control argument, *cstring*. This is a string which is for the most part just PRINC'd, except that the character "~" starts an escape sequence which directs other actions. A "~" escape sequence has an (optional) numeric parameter followed, optionally, by either or both of the modifiers @ and :, followed finally by a mode character. These escape actions can use up one or more of the non-control arguments.

Possible escape sequences are:

| Command | Interpretation |
|---|---|
| ~% | Insert a newline. |
| ~*n*% | Insert n newlines. |
| ~& | Get a fresh line (ie, newline iff not in col 0) |
| ~M | Print an expression in display (output) form. |
| ~:M | Print an expression in linear (input) form. |
| ~@M | Print an expression in operand (eg, MTIMES → &*) form. |
| ~:@M | Combination of ":" and "@" modifiers. |
| ~A | Lisp PRINC the object. |
| ~S | Lisp PRINT the object. |
| ~<cr> | Ignore the <cr> and leading whitespace on next line. |

Sample Usage:

```
(MFORMAT T "~%This is line 1.~
           and this is line 2")
This is line 1
and this is line 2
()
(MFORMAT T "~%The expression ~:M was used." '((MEXPT) $A $B))
The expression A↑B was used.
()
```

PRINL

(PRINL *l*)

[1 arg] Uses PRINC to type each of the elements of *l* (which must be a Lisp list) separated by spaces.

# Error System

MERROR

(MERROR *message* $q_1$ $q_2$ ...) *Special Form*

*message* should be a control-string which can be fed to MFORMAT and args are the arguments to be used by the control-string.
MERROR will do a FRESH-LINE operation before calling MFORMAT and a FRESH-LINE afterward, so for single line error messages with no arguments there is no need to put any "~&" or "~%" in the control-string.

Sample Usage:

```
;; One might implement FEXPRCHK as follows
(DEFUN FEXPRCHK (L FN)
  (IF (OR (NULL L) (CDR L))
      (MERROR "Wrong number of args to ~:@M~%~M" FN L)))
```

# Argument Checking

## FEXPRCHK

(FEXPRCHK *arglist fname*)

[2 args] Checks to see that *arglist* (which should be the argument list received by some fexpr, whose name is *fname*) contains exactly one element. If it does not, an error is signalled.

## NONSYMCHK

(NONSYMCHK *obj fname*)

[2 args] Checks to see that *obj* is an atomic symbol. If not, an error is signalled, citing a call to the function *fname*.

## NONVARCHK

(NONVARCHK *obj fname consterrp secondp*)

[4 args] Does error checking for a non-variable (ie, signals an error if *obj* is not variable). If *consterrp* is non-(), also signals an error for atomic constants. If an error is signalled, it will refer to a call to *fname* and if *secondp* is non-(), will refer explicitly to the second argument to that function (eg, for INTEGRATE, DIFF, LIMIT, ...).

ERLIST

(ERLIST *message*) *Special Form*

Takes an arg of a *message*. Atomic messages are typed out with PRINC; lists are typed out with PRINL. Then an error is signalled.
This form is obsolete. Use MERROR.

ERREXP

(ERREXP)

[No args] Types out "ERREXP contains the offending expression." Note that ERREXP must be SETQ'd 'by hand'. This function does not do that for you.

# PNames

# Explosives

## MEXPLODEN

(MEXPLODEN $x$)

[1 arg] Like Lisp's EXPLODEN function, but binds BASE to 10 and *NOPOINT to T during the process.

## MSTRING

(MSTRING $x$)

[1 arg] A Macsyma-flavored EXPLODEN function which works on arbitrary expressions except MTEXT and MLABLE. Returns a list of the fixnum characters that make up the expressions strung out representation.

Examples:

```
(MSTRING '((MPLUS SIMP) $X $Y)) → (#/X #/+ #/Y)
(MSTRING '(($F) $A))            → (#/F #/( #/A #/))
```

# Explosives (Cont'd)

## STRING*

(STRING* *x*)

[1 arg] A Macsyma-flavored EXPLODEN function - for atomic objects only. Returns the list of fixnums that make up the Macsyma printname of the object. Doesn't handle bigfloats.
Doesn't bind BASE or *NOPOINT. Doesn't do error checking for non-atoms; result in such case is garbage, but no error is generated.

Examples:

```
(STRING* '$FOO)         → (#/F #/O #/O)
(STRING* '$SUBSTITUTE) → (#/S #/U #/B #/S #/T)
```

## STRING*1

(STRING*1 *x*)

[1 arg] A faster version of STRING*. Expects that NUMBERP and ALIASLIST checks have already been done. Doesn't handle bigfloats.
Doesn't bind BASE or *NOPOINT. Doesn't do error checking for non-atoms; result in such case is garbage, but no error is generated.

## MAKSTRING*

(MAKSTRING* *x*)

[1 arg] Same as STRING* but returns a list of atomic symbols instead of fixnums.

Example:

```
(MAKSTRING* '$FOO) → (F O O)
```

# Prefix Character Control

## DOLLARIFY

(DOLLARIFY *l*)

[1 arg] The argument *l* should be a Lisp list containing only symbols. Returns a SIMP'd Macsyma list of dollarsigned symbols.

Example:

(DOLLARIFY '(A B C)) → ((MLIST SIMP) $A $B $C)

## STRIPDOLLAR

(STRIPDOLLAR *x*)

[1 arg] Turn a lisp atom or BIGFLOAT form into its Macsyma display form. Strips the "$", "%" or "&" (if any) from the head of the printname of symbols, turns ((BIGFLOAT ...) ...) into a symbol whose printname is the bigfloat's more pretty form.

Examples:

```
(STRIPDOLLAR '((BIGFLOAT SIMP 56) 54975581388800000 17)) → |1.0B5|
(STRIPDOLLAR '$FOO)                                      → FOO
(STRIPDOLLAR '%FOO)                                      → FOO
(STRIPDOLLAR '&FOO)                                      → FOO
(STRIPDOLLAR 'MEXPT)                                     → MEXPT
(STRIPDOLLAR '$$FOO)                                     → $FOO
```

# Prefix Character Control (Cont'd)

## FULLSTRIP1

(FULLSTRIP1 *sym*)

[1 arg] Remove dollarsign (like STRIPDOLLAR) and also do any inverse aliasing according to defined ALIAS's.

Examples:

```
(FULLSTRIP1 '$FOO)          → FOO
(FULLSTRIP1 '$SUBSTITUTE) → SUBST
```

## FULLSTRIP

(FULLSTRIP *l*)

[1 arg] Calls FULLSTRIP1 on each element of its arg, *l*, which must be a list.

Example:

```
(FULLSTRIP '($A $B $C)) → (A B C)
```

# Operators/Operands

## GETOP

(GETOP *x*)

[1 arg] Gets the operand form of an operator (the OP property). If *x* is not a symbol or has no operand form, *x* is just returned.

Examples:

```
(GETOP 'MPLUS) → &+
(GETOP '&+)    → &+
(GETOP '%EXPT) → %EXPT
(GETOP '$FOO)  → $FOO
```

## GETOPR

(GETOPR *x*)

[1 arg] Gets the operator form of an operand (the OPR property). If *x* is not a symbol or has no operator form, *x* is just returned.

Examples:

```
(GETOP 'MPLUS) → MPLUS
(GETOP '&+)    → MPLUS
(GETOP '%EXPT) → %EXPT
(GETOP '$FOO)  → $FOO
```

# File System

# File Names

## FILESTRIP

(FILESTRIP *filespec*)

[1 arg] Converts a Macsyma file specification into a Lisp file specification. *filespec* should be a Lisp list of symbols.
Does not change DEFAULTF.

Examples:

```
(DEFAULTF '((DSK FOO) FILE >))      → ((DSK FOO) FILE >)
(FILESTRIP '(&BAR/;))               → (* > DSK BAR)
(FILESTRIP '(&BAR/ BAZ))            → (BAR BAZ DSK FOO)
(FILESTRIP '($A $B $C $D))          → (A B C D)
(FILESTRIP '($A $B $C))             → (A B DSK C)
(FILESTRIP '($A $B $DSK))           → (A B DSK FOO)
(FILESTRIP '($SUBSTITUTE MEXPT)) → (SUBST MEXPT DSK FOO)
```

## MTRUENAME

(MTRUENAME *fileobj*)

[1 arg] Returns the dollarified Macsyma filespec (as an mlist) which is the truename of *fileobj*.

Examples:

```
(MTRUENAME (OPEN '((DSK FOO) BAR >) 'OUT)) → ((MLIST SIMP) $BAR $1 $DSK $FOO)
(MTRUENAME TYO)                            → ((MLIST SIMP) $* $* $TTY $*)
```

## TRUEFNAME

(TRUEFNAME *filespec*)

[1 arg] Gets the TRUENAME of *filespec*, which should be specified in lisp *filespec* format.

Examples:

```
(TRUEFNAME '((DSK MAXDOC) UPDATE >)) → ((DSK ALJABR) UPDATE /298)
(TRUEFNAME '(UPDATE > DSK MAXDOC))   → ((DSK ALJABR) UPDATE /298)
```

# Mathy Functions

# Equality

## ALIKE1

(ALIKE1 $x$ $y$)

[1 arg] Predicate returns T if Macsyma objects $x$ and $y$ are equivalent. Use this as you would use EQUAL on Lisp objects. Compares everything in the object except the flags in the CDAR (expression property list) of lists (it does notice though if one object is an ARRAY and another isn't and will return ( ) in that case – all other flags are ignored).

# Predicates

## FREE

(FREE *exp var*)

[2 args] Checks to see if *exp* is free of *var*. Both args are assumed to be Macsyma expressions in general representation.

## $BFLOATP

($BFLOATP *x*)

[1 arg] Returns T if its arg is a bigfloat number.

# Evaluation/Simplification

# Evaluation

## $EV

*Special Form* ($EV *mexp* $spec_1$ $spec_2$)

This invokes the Macsyma evaluator with evaluation control given by $spec_1$, $spec_2$, etc.
Should NEVER be used in lisp code.

## MEVAL

(MEVAL *mexp*)

[1 arg] A Macsyma flavored EVAL. Does both evaluation and simplification.
Any call to the macsyma evaluator not within the evaluator itself has far-ranging effects on the entire system. It should not be used for trivial functionalities such as substitution and simplification. This information is only provided for the sake of reading old code.

Example:

(MEVAL '((MPLUS) 2 2)) → 4

Equivalent to:

(SIMPLIFYA (MEVAL1 FORM) ())

## MEVAL1

(MEVAL1 *mexp*)

[1 arg] A Macsyma flavored EVAL that does only evaluation. No final simplification is done on the result.

Example:

(MEVAL1 '((MPLUS) 2 2)) → ((MPLUS) 2 2)

# Funarg Application

## MAPPLY

(MAPPLY *mfun arglist fname*)

[3 args] A Macsyma flavored APPLY. Applies *mfun* to *arglist*. *fname* is used as the function name to refer to if an error occurs during the application.

## MCALL

(MCALL *mfun arglist fname*)

[1 or more] A Macsyma flavored FUNCALL. Calls *mfun* on the given args. Assumes *mfun* is a valid name to type out if an error occurs, so *mfun* should be a symbol.
(MCALL *mfun* $arg_1$ $arg_2$ ...) = (MAPPLY *mfun* (LIST $arg_1$ $arg_2$ ...) *mfun*).

## MBIND

(MBIND *vars fnargs fnname*)

[3 args] Involved with Macsyma variable "binding". *** What does this do? ***

## MUNBIND

(MUNBIND *vars*)

[1 arg] Involved with Macsyma variable "unbinding". *** What does this do? ***

# Simplification

## SIMPLIFY

(SIMPLIFY *mexp*)

[2 args] Invokes the Macsyma expression simplifier on *mexp*, returning a simplified expression.

Examples:

```
(SIMPLIFY '((MPLUS) 2 2))      → 4
(SIMPLIFY '((MTIMES) $A $A)) → ((MEXPT SIMP) $A 2)
```

Equivalent to:

(SIMPLIFYA *mexp* ())

## SIMPLIFYA

(SIMPLIFYA *mexp flag*)

[2 args] Simplifies *mexp*. If *flag* is non-(), the arguments to *mexp* are assumed to be simplified already and their simp flags are not checked. If *flag* is (), this assumption is not made and the args are checked for simp flags and simplified if no such flags are seen. When in doubt, use a *flag* of (). (See SIMPLIFY.)

Examples:

```
(SIMPLIFYA '((MPLUS) 2 2) ())      → 4
(SIMPLIFYA '((MTIMES) $A $A) ()) → ((MEXPT SIMP) $A 2)
```

# Miscellania

# Information

## KILL1

(KILL1 *sym*)

[1 arg] The subr form of Macsyma's KILL command. Kills *sym*, which must be a symbol, by doing a MAKUNBOUND and removing all Macsyma-related properties. A number of keywords are available (eg, FUNCTIONS, VALUES, ALLBUT(...), ALL, ...). For details, see the code.

## ADD2LNC

(ADD2LNC *obj mlist*)

[2 args] If *obj* is in the given *mlist* already (according to a MEMALIKE test), it is spliced onto the end of the list with NCONC. If *obj* is already in *mlist*, no change occurs.

# Nouns/Verbs

## $NOUNIFY

(\$NOUNIFY *sym*)

[1 arg] Returns the noun form of *sym*, which must be a symbol.
If the symbol has a VERB property, uses that. If the thing is a NOUN, it returns its arg. Else strips a leading "M" or "$" from the printname and puts a "%" on. Puts the verb form in the NOUN property of the verb and the noun in the VERB property of the noun.

Examples:

```
($NOUNIFY '$A)     → %A
($NOUNIFY '%A)     → %A
($NOUNIFY 'MEXPT)  → %EXPT
```

## $VERBIFY

(\$VERBIFY *sym*)

[1 arg] Returns the verb form of *sym*, which must be a symbol.
If the symbol has a noun property, uses that. If the arg is a VERB or doesn't start with "%", just returns it. Else strips the "%", adds a "$". Calls $NOUNIFY to set up all the VERB/NOUN properties, and returns what's in the thing's NOUN property.

Examples:

```
($VERBIFY '$A)     → $A
($VERBIFY '%A)     → %A
($VERBIFY '%EXPT)  → MEXPT
```

# Predicates

## ALPHABETP

(ALPHABETP *x*)

[1 arg] Predicate returns T if its arg is a fixnum representing an alphabetic character or is in an exceptional character which is a member of the list contained in the variable ALPHABET.

Examples:

(ALPHABETP #/A) → T
(ALPHABETP #/%) → T

# MList Hacking

## $LISTP

(\$LISTP $x$)

[1 arg] Returns T if its arg is an mlist.

Examples:

```
($LISTP '(FOO BAR))            → ()
($LISTP '((MLIST) $FOO $BAR)) → T
($LISTP '((MLIST SIMP)))       → T
($LISTP ())                    → ()
```

## $CONS

(\$CONS $x$ $l$)

[2 args] A Macsyma flavored CONS.
Makes a Macsyma-style cons, doing the right thing with the (MLIST) in the car of the of the list, $l$.

Examples:

```
($CONS '$A '((MLIST)))          → ((MLIST) $A)
($CONS '$A '((MLIST FOO) $B)) → ((MLIST FOO) $A $B)
($CONS '$A '$B)                 → error!
```

# MList Hacking (Cont'd)

## $FIRST

($FIRST *mexp*)

[1 arg] A Macsyma flavored CAR.
Returns the first element of the Macsyma expression, *mexp*.

Examples:

```
($FIRST '((MLIST) $A $B))  → $A
($FIRST '((MPLUS) $A $B)) → $A
($FIRST '((MLIST)))        → error!
```

## $REST

($REST *x* [*n*])

[1 or 2 args] A Macsyma flavored CDR.
If $n$ is supplied, then if it is positive the first $n$ elements are skipped and the rest returned; if $n$ is negative, then the last $-n$ elements are ignored and the rest returned.

(Removes any flags from the cdar of the form that was its arg, but doesn't care if its arg wasn't an mlist.)

Examples:

```
($REST '((MLIST) $A $B $C))     → ((MLIST) $B $C)
($REST '((MLIST) $A $B $C) 2)   → ((MLIST) $C)
($REST '((MPLUS SIMP) $A $B)) → ((MPLUS) $B)
```

# Index

# *Index*


$BFLOATP . . . 34
$CONS . . . 43
$EV . . . 36
$FIRST . . . 44
$LISTP . . . 43
$NOUNIFY . . . 41
$REST . . . 44
$VERBIFY . . . 41
*MAKE-LIST . . . 3
A-ATOM . . . 11
ADD2LNC . . . 40
ALIKE1 . . . 33
ALPHABETP . . . 42
AMONG . . . 7
AMONGL . . . 7
ANDMAPC . . . 5
ANDMAPCAR . . . 5
ASSOL . . . 15
ASSOLIKE . . . 15
ASSQR . . . 10
C-PUT . . . 11
CONCAT . . . 13
DISPLA . . . 17
DOLLARIFY . . . 27
DOT2L . . . 10
ERLIST . . . 23
ERREXP . . . 23
FEXPRCHK . . . 22
FILESTRIP . . . 31
FIRSTN . . . 9
FLOAT-GENCALL . . . 37
FREE . . . 34
FSET . . . 2
FULLSTRIP . . . 28
FULLSTRIP1 . . . 28
GETOP . . . 29
GETOPR . . . 29
KILL1 . . . 40
LOG2 . . . 12
MAKSTRING* . . . 26
MAP2C . . . 4
MAPPLY . . . 37
MBIND . . . 37
MCALL . . . 37
MEMALIKE . . . 15
MERROR . . . 21

MEVAL . . . . . 36
MEVAL1 . . . . . 36
MEXPLODEN . . . . . 25
MFORMAT . . . . . 18
MSTRING . . . . . 25
MTRUENAME . . . . . 31
MUNBIND . . . . . 37
NONSYMCHK . . . . . 22
NONVARCHK . . . . . 22
NXTTOLAST . . . . . 9
ORMAPC . . . . . 5
PRINL . . . . . 19
RECONC . . . . . 8
SIMPLIFY . . . . . 38
SIMPLIFYA . . . . . 38
STRING* . . . . . 26
STRING*1 . . . . . 26
STRIPDOLLAR . . . . . 27
TRUEFNAME . . . . . 31
XOR . . . . . 6